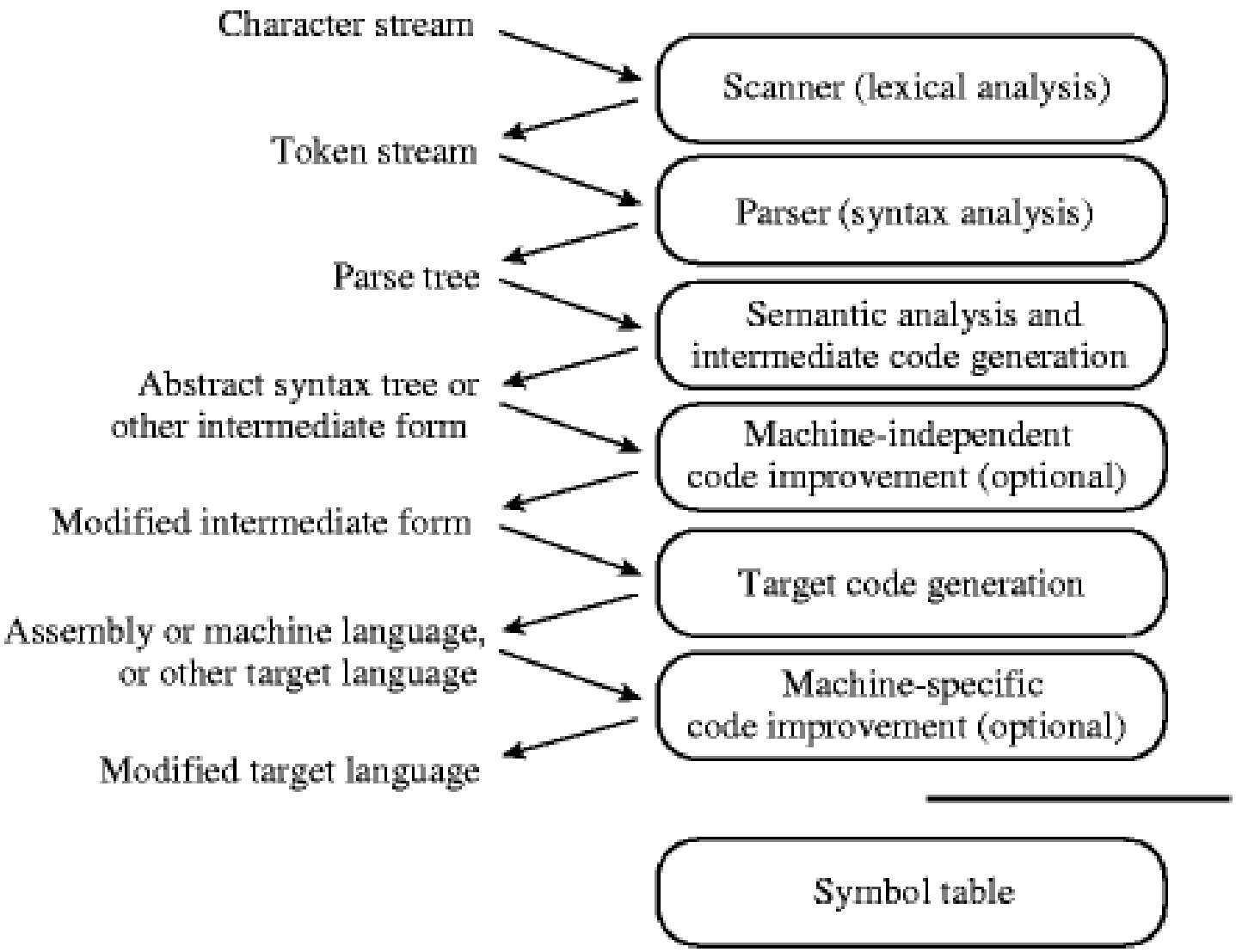
WEEK 2

DESCRIBING SYNTAX

Phases of Compilation



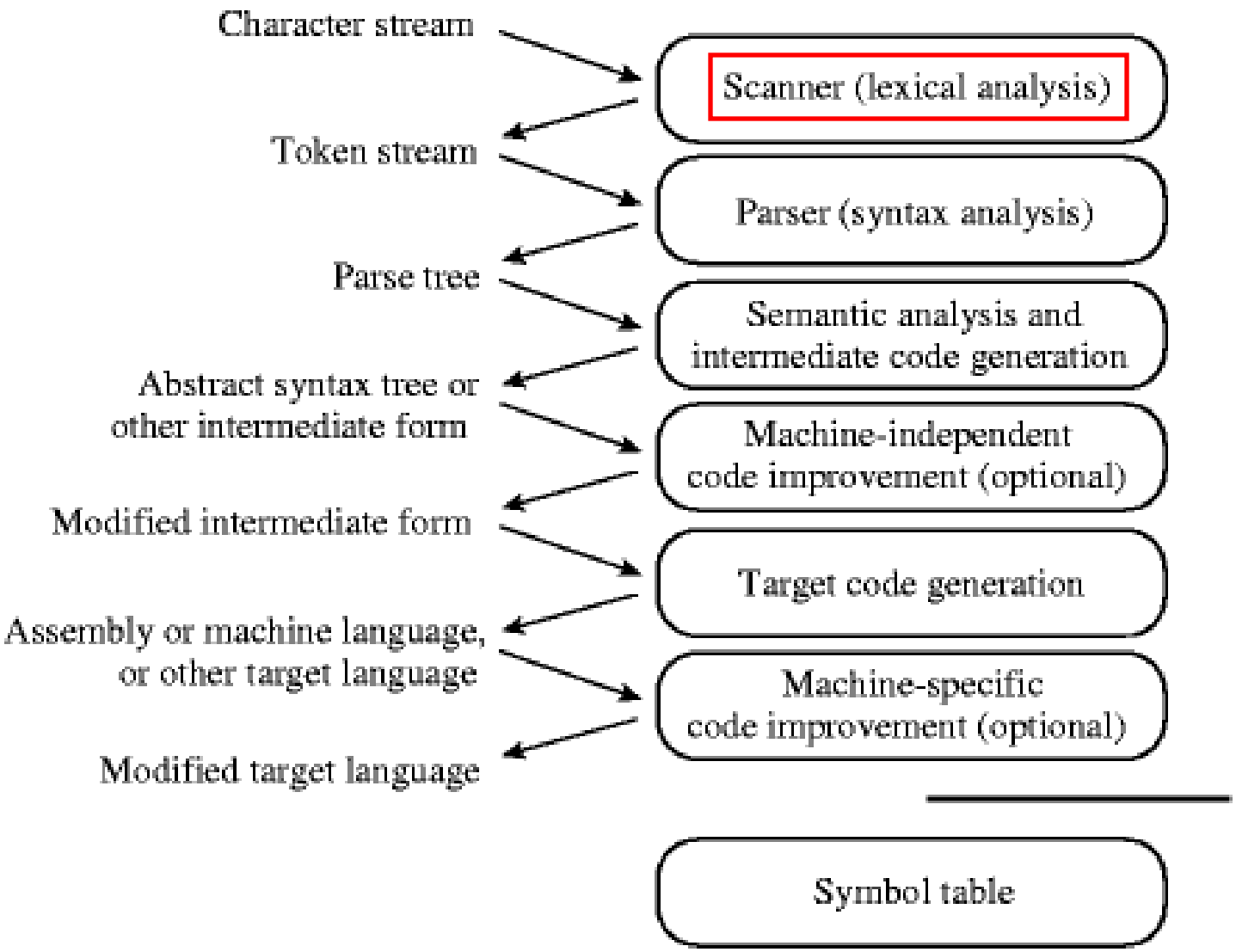
Specification of Programming Languages

- PLs require precise definitions (i.e. no ambiguity)
  - Language *form* (Syntax)
  - Language *meaning* (Semantics)
- Consequently, PLs are specified using formal notation:
  - Formal syntax
    - Tokens
    - Grammar
  - Formal semantics

Specification of Programming Languages

- **Syntax**: the form or structure of the expressions, statements, and program units
  - `while (boolean-expr) statement`
- **Semantics**: the meaning of the expressions, statements, and program units
  - when the current value of boolean-expr is true, the embedded statement is executed, …
- Syntax and semantics provide a language's definition
  - Users of a language definition
    - Other language designers
    - Implementers
    - Programmers (the users of the language)

Phases of Compilation



Scanner

- Main task: identify tokens
  - Basic building blocks of programs
  - *e.g.* keywords, identifiers, numbers, punctuation marks
- Desk calculator language example:

```
read A
sum := A + 3.45e-3
write sum
write sum / 2
```

Formal Definition of Tokens

- A set of tokens is a set of strings over an alphabet
  - {read, write, +, −, *, /, :=, 1, 2, …, 10, …, 3.45e−3, …}
- A set of tokens is a *regular set* that can be defined by comprehension using a *regular expression*
- For every regular set, there is a *deterministic finite automaton* (DFA) that can recognize it
  - i.e. determine whether a string belongs to the set or not
  - Scanners extract tokens from source code in the same way DFAs determine membership

Regular Expressions

- To specify tokens, we use regular expressions
- A regular expression is one of the following:
  - A single character
  - The empty string, denoted by ϵ
  - The concatenation of two regular expressions
    - *Notation:* RE_1 RE_2 (*i.e.* RE_1 followed by RE_2)
  - The union of two regular expressions
    - *Notation:* RE_1 | RE_2
  - The closure of a regular expression
    - *Notation:* RE*
    - * is known as the *Kleene star*
    - * represents the concatenation of 0 or more strings

Regular Expressions

- Numerical constants accepted by a simple hand-held calculator:

$$
\begin{aligned}
\textit{number} &\longrightarrow \textit{integer} \mid \textit{real} \\
\textit{integer} &\longrightarrow \textit{digit}\ \textit{digit}^* \\
\textit{real} &\longrightarrow \textit{integer}\ \textit{exponent} \mid \textit{decimal}\ (\textit{exponent} \mid \epsilon) \\
\textit{decimal} &\longrightarrow \textit{digit}^*\ (\ .\ \textit{digit} \mid \textit{digit}\ .\ )\ \textit{digit}^* \\
\textit{exponent} &\longrightarrow (\texttt{e} \mid \texttt{E})\ (\texttt{+} \mid \texttt{-} \mid \epsilon)\ \textit{integer} \\
\textit{digit} &\longrightarrow \texttt{0} \mid \texttt{1} \mid \texttt{2} \mid \texttt{3} \mid \texttt{4} \mid \texttt{5} \mid \texttt{6} \mid \texttt{7} \mid \texttt{8} \mid \texttt{9}
\end{aligned}
$$

- Recursion is not allowed!
- Notice the use of parentheses to avoid ambiguity

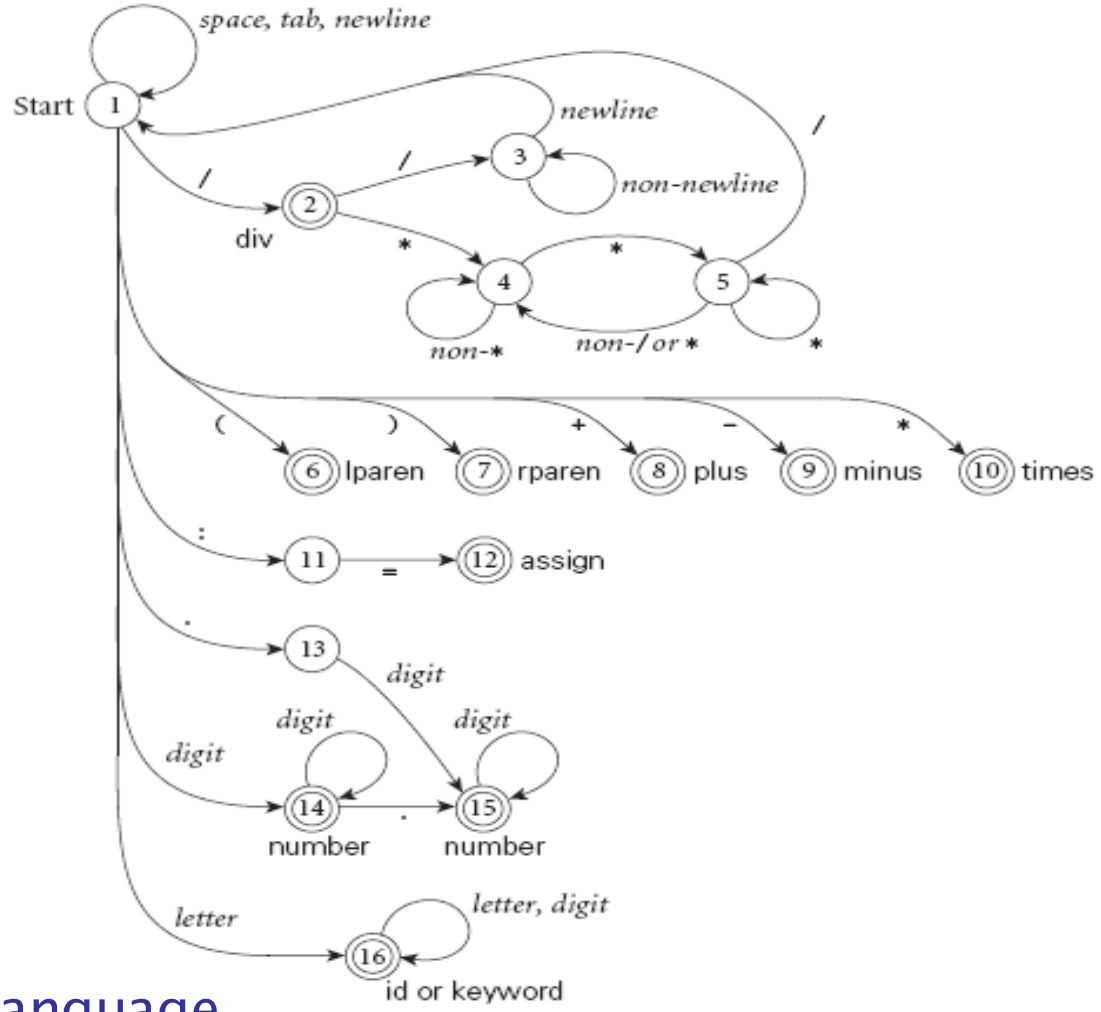
Scanning

Pseudocode of the scanner for calculator language

```
skip any initial white space (spaces, tabs, and newlines)
if cur_char ∈ { '(', ')', '+', '-', '*' }
    return the corresponding single-character token
if cur_char = ':'
    read the next character
    if it is '=' then return assign else announce an error
if cur_char = '/'
    peek at the next character
    if it is '*' or '/'
        read additional characters until "*/" or newline is seen, respectively
        jump back to top of code
    else return div
if cur_char = .
    read the next character
    if it is a digit
        read any additional digits
        return number
    else announce an error
if cur_char is a digit
    read any additional digits and at most one decimal point
    return number
if cur_char is a letter
    read any additional letters and digits
    check to see whether the resulting string is read or write
    if so then return the corresponding token
    else return id
else announce an error
```

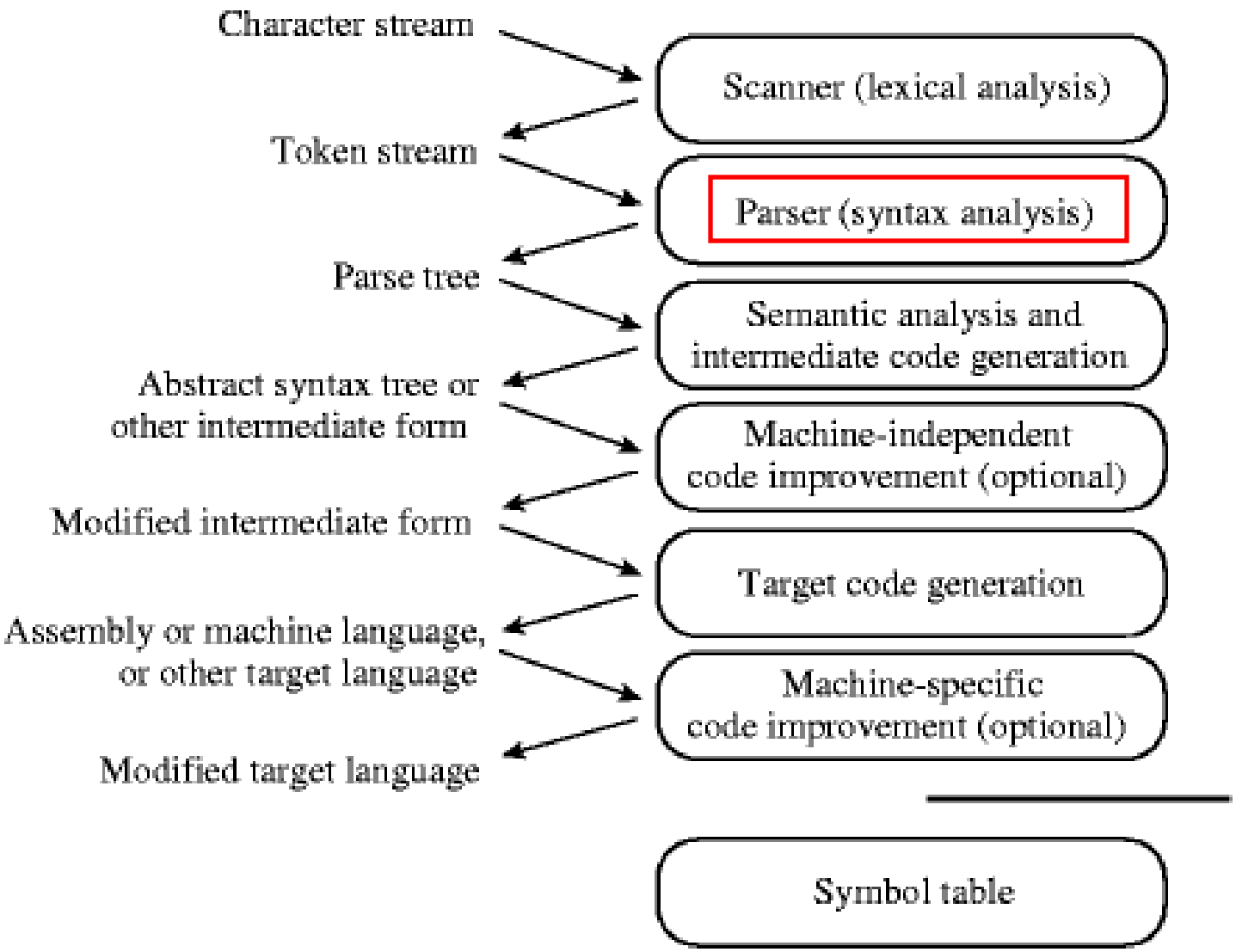
Deterministic Finite Automaton

scanner for calculator tokens in the form of a finite automaton



- Regular expressions "generate" a regular language
- DFAs "recognize" it

Phases of Compilation



The General Problem of Describing Syntax: Terminology

- A *sentence* is a string of characters over some alphabet
- A *language* is a set of sentences
- A *lexeme* is the lowest level syntactic unit of a language (e.g., `*`, `sum`, `begin`)
- A *token* is a category of lexemes (e.g., identifier, keyword, number, string, ...)

Syntax Analysis

- The syntax of a programming language
  - Describes its form
    - *i.e.* **Organization of tokens** *(elements)*
  - Formal notation
    - Context Free Grammars (CFGs)

BNF and Context-Free Grammars

Context-Free Grammars

- Developed by Noam Chomsky in the mid-1950s
- Language generators, meant to describe the syntax of natural languages
- Define a class of languages called context-free languages

Backus-Naur Form (1959)

- Invented by John Backus to describe the syntax of Algol 58
- BNF is equivalent to context-free grammars

Context-Free Grammars

- Add recursion to regular expressions
  - Nested constructions
- Notation

expression → *identifier* | *number* | **-** *expression*
| **(** *expression* **)**
| *expression operator expression*

operator → **+** | **-** | ***** | **/**

  - **Terminal symbols**
  - *Non-terminal symbols*
  - *Start symbol (a non-terminal)*
  - Production rule (i.e. substitution rule)

terminal symbol → terminal and non-terminal symbols

Backus-Naur Form

- Backus-Naur Form (BNF)
  - Equivalent to CFGs in power
  - CFG

```
expression → identifier | number | - expression
           | ( expression )
           | expression operator expression
operator → + | - | * | /
```

  - BNF

```
⟨expression⟩ → ⟨identifier⟩ | ⟨number⟩ | - ⟨expression⟩
             | ( ⟨expression⟩ )
             | ⟨expression⟩ ⟨operator⟩ ⟨expression⟩
⟨operator⟩ → + | - | * | /
```

BNF Fundamentals

- In BNF, abstractions are used to represent classes of syntactic structures (also called *nonterminal symbols*, or just *terminals*)
- *Terminals* are lexemes or tokens
- A rule has a left-hand side (LHS), which is a nonterminal, and a right-hand side (RHS), which is a string of terminals and/or nonterminals

BNF Fundamentals (continued)

- *Nonterminals* are often enclosed in angle brackets
  - Examples of BNF rules (Java if statement):

```
<ident_list> → identifier | identifier, <ident_list>
<if_stmt> → if (<logic_expr>) <stmt>
          | if (<logic_expr>) <stmt> else <stmt>
```

- Grammar: a finite non-empty set of rules
- A *start symbol* is a special element of the nonterminals of a grammar

BNF Rules

- A derivation is a repeated application of rules, starting with the start symbol and ending with a sentence (all terminal symbols)

- An abstraction (or nonterminal symbol) can have more than one RHS

```
<stmt> → <single_stmt>
       | begin <stmt_list> end
```

Describing Lists

- Syntactic lists are described using recursion

```
<ident_list> → ident
             | ident, <ident_list>
```

A Grammar for a Small Language

```
<program>   → begin <stmt_list> end
<stmt_list> → <stmt> | <stmt> ; <stmt_list>
<stmt>      → <var> = <expr>
<var>       → A | B | C
<expr>      → <var> + <var> | <var> - <var> | <var>
```

An Example Derivation

```
<program>   => begin <stmt_list> end
            => begin <stmt> ; <stmt_list> end
            => begin <var> = <expr> ; <stmt_list> end
            => begin A = <expr> ; <stmt_list> end
            => begin A = <var> + <var> ; <stmt_list> end
            => begin A = B + <var> ; <stmt_list> end
            => begin A = B + C ; <stmt_list> end
            => begin A = B + C ; <stmt> end
            => begin A = B + C ; <var> = <expr> end
            => begin A = B + C ; B = <expr> end
            => begin A = B + C ; B = <var> end
            => begin A = B + C ; B = C end
```

Derivations

- Every string of symbols in a derivation is a *sentential form*
- A *sentence* is a sentential form that has only terminal symbols
- A *leftmost derivation* is one in which the leftmost nonterminal in each sentential form is the one that is expanded
- A derivation may be neither leftmost nor rightmost

Another Example Derivation

Grammar for Simple Assignment Statements:

```
<assign>    → <id> = <expr>
<id>        → A | B | C
<expr>      → <id> + <expr> | <id> * <expr> | ( <expr> ) | <id>
```

Leftmost Derivation: A = B * (A + C)

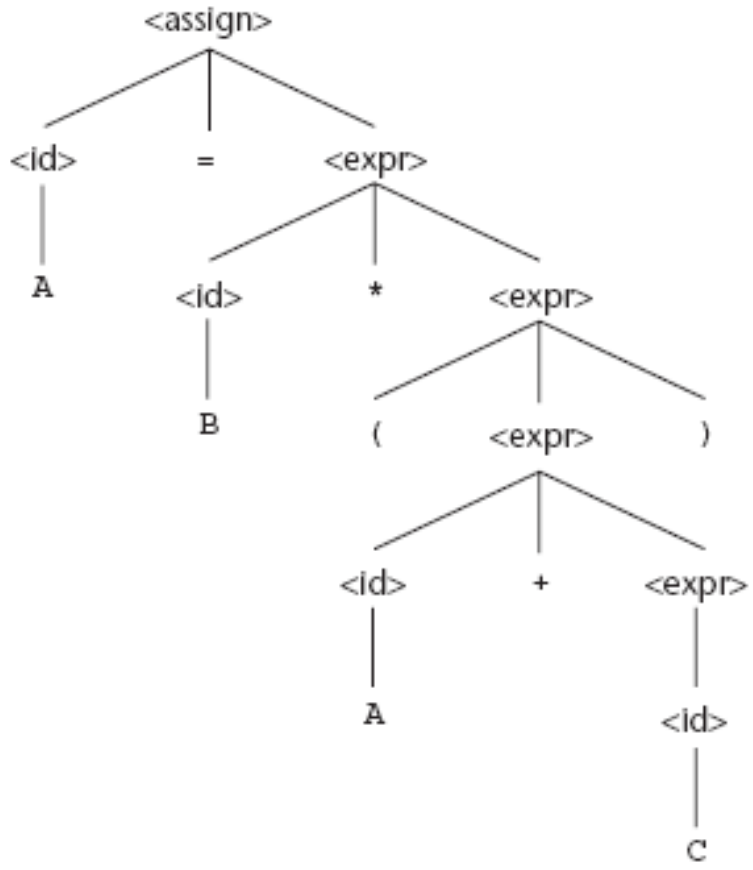
```
<assign>    => <id> = <expr>
            => A = <expr>
            => A = <id> * <expr>
            => A = B * <expr>
            => A = B * ( <expr> )
            => A = B * ( <id> + <expr> )
            => A = B * ( A + <expr> )
            => A = B * ( A + <id> )
            => A = B * ( A + C )
```

Parse Tree

- A graphical representation of a derivation

Figure 3.1

A parse tree for the simple statement
`A = B * (A + C)`



Ambiguity in Grammars

- A grammar is *ambiguous* if and only if it generates a sentential form
  - that has two or more distinct parse trees

An Ambiguous Expression Grammar

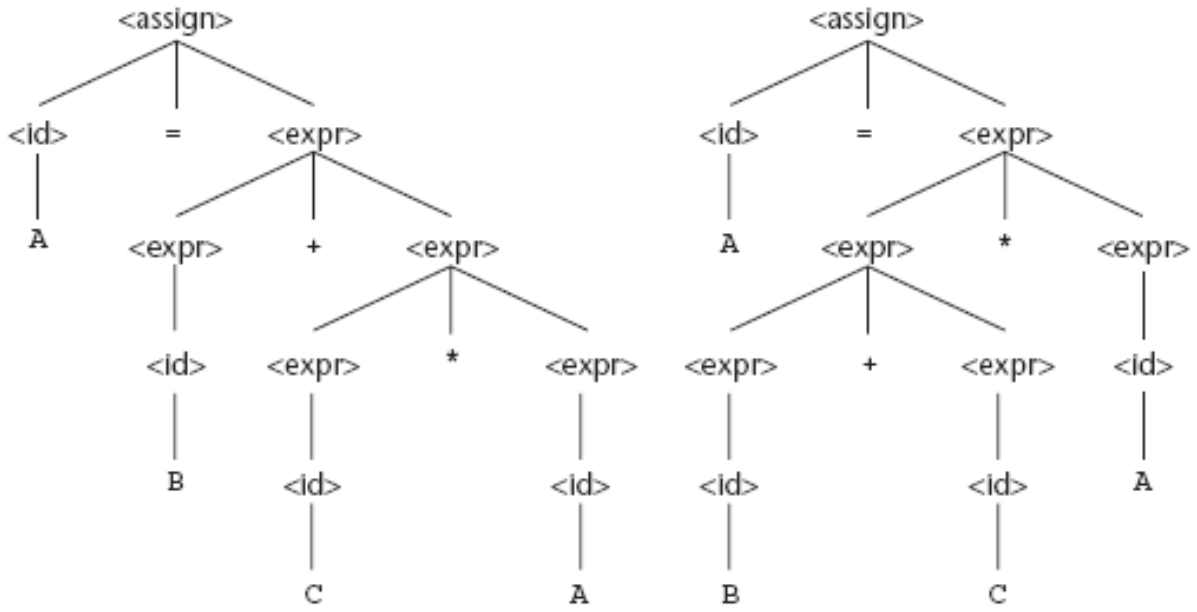
<assign> → <id> = <expr>

<id> → A | B | C

<expr> → <expr> + <expr> | <expr> * <expr> | (<expr>) | <id>

Figure 3.2

Two distinct parse trees for the same sentence, A = B + C * A



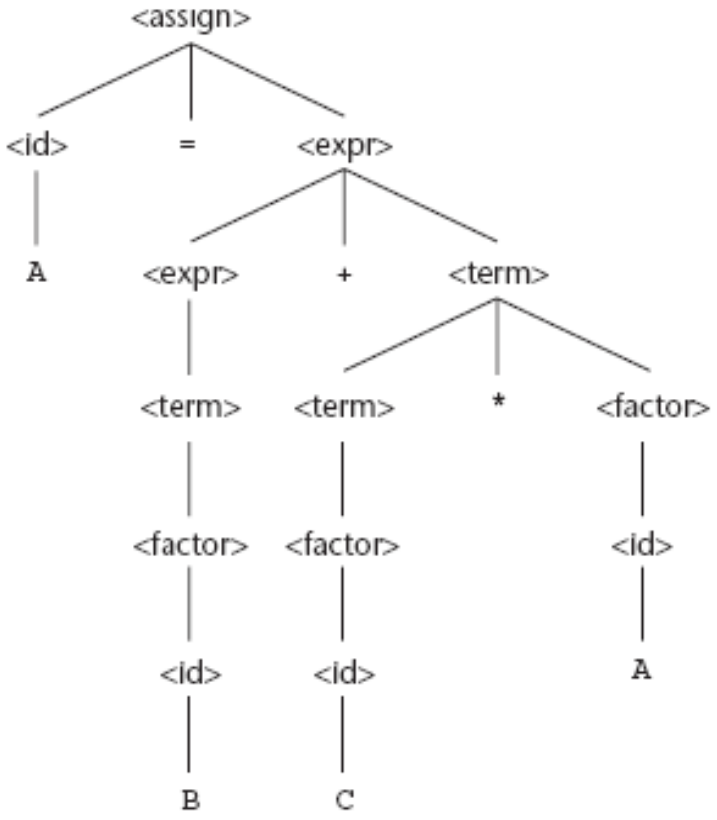
An Unambiguous Expression Grammar

- If we use the parse tree to indicate
  - precedence levels of the operators,

  we cannot have ambiguity

- Example: left associativity and operator precedence

  B + C * A means B + (C * A)

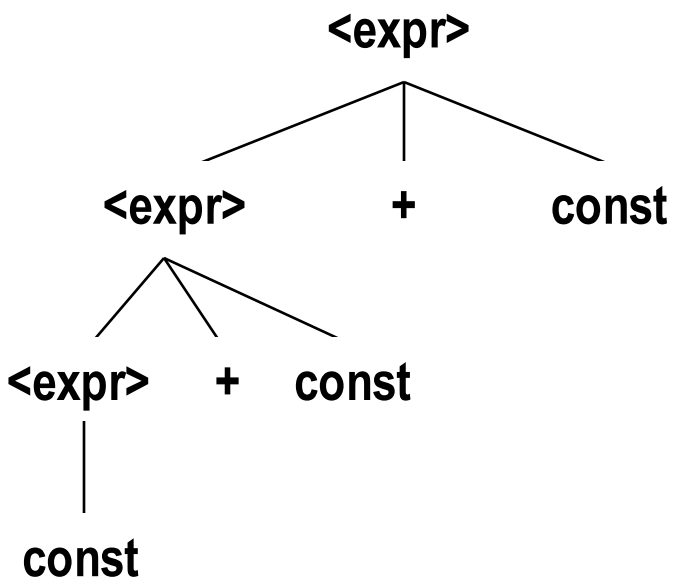



```
<assign> → <id> = <expr>
<id>     → A | B | C
<expr>   → <expr> + <term> | <term>
<term>   → <term> * <factor> | <factor>
<factor>     → ( <expr> ) | <id>
```

Associativity of Operators

- Operator associativity can also be indicated by a grammar

```
<expr> -> <expr> + <expr> |  const  (ambiguous)
<expr> -> <expr> + const  |  const  (unambiguous)
```



Unambiguous Grammar for Selector

- Java `if-then-else` grammar

```
<stmt> -> <if_stmt>
<if_stmt> -> if (<logic_expr>) <stmt>
           | if (<logic_expr>) <stmt> else <stmt>
```

Ambiguous!

- An unambiguous grammar for `if-then-else`

```
<stmt>      -> <matched> | <unmatched>
<matched>   -> if (<logic_expr>) <matched> else <matched>
             | a non-if statement
<unmatched> -> if (<logic_expr>) <stmt>
             | if (<logic_expr>) <matched> else <unmatched>
```

Extended BNF

- Optional parts are placed in brackets [ ]

```
<stmt_list> → <stmt> | <stmt> ; <stmt_list>  (BNF)
<stmt_list> → <stmt> [ ; <stmt_list> ]       (EBNF)
```

- Alternative parts of RHSs are placed inside parentheses and separated via vertical bars

```
<expr>   → <var> + <var> | <var> - <var>  (BNF)
<expr>   → <var> (+ | -) <var>            (EBNF)
```

- Repetitions (0 or more) are placed inside braces { }

```
<term>   → <term> * <factor> | <factor>   (BNF)
<term>   → <factor> {* <factor>}          (EBNF)
```

Extended BNF (summary table)

META-SYMBOL	FUNCTION	DESCRIPTION
(and)	grouping	used to group syntactic constructions
[and]	option	encloses an optional construction
{ and }	repetition	encloses a construction that may be repeated zero or more times
|	alternative	separates alternative constructions
→	production	separates the left and right hand sides of a production

BNF and EBNF

- BNF

```
<expr>  → <term> + <term>
        | <term> - <term>
        | <term>
<term>  → <factor> * <factor>
        | <factor> / <factor>
        | <factor>
```

- EBNF

```
<expr>  → <term> {(+ | -) <term>}
<term>  → <factor> {(* | /) <factor>}
```

Recent Variations in EBNF

- Alternative RHSs are put on separate lines
- Use of a colon instead of $\rightarrow$
- Use of $_{\texttt{opt}}$ subscript for optional parts
- Use of `oneof` for choices

- Use of $^{+}$ superscript to indicate one or more repetitions